

Absolutely. Below is a detailed, beginner-friendly study/training material for **Git Fundamentals: Repositories, Commits, Branches, and Merges**, with simple explanations, commands, and practical examples.

Git Fundamentals: Repositories, Commits, Branches & Merges

1. What is Git?

Git is a **Version Control System (VCS)** used to track changes in files and source code.

In simple words:

Git helps you save different versions of your project so you can see what changed, go back to an older version, and work with other developers safely.

Example

Suppose you are creating a website.

On Monday:

```
index.html
style.css
script.js
```

You make some changes on Tuesday and accidentally break the website.

Without Git, you may have difficulty getting back to Monday's working version.

With Git, you can save versions:

```
Version 1 → Basic website
Version 2 → Added login page
Version 3 → Added dashboard
Version 4 → Fixed login bug
```

You can inspect or restore previous versions when needed.

2. Why Do We Need Git?

Git is useful because it allows us to:

- Track changes in files
- Save different versions of a project

- See who changed something
- See what was changed
- Undo mistakes
- Work on multiple features independently
- Combine work from multiple developers
- Collaborate using platforms such as GitHub

Real-world example

Imagine 3 developers working on an application:

```
Developer A → Login
Developer B → Payment
Developer C → User Profile
```

They can work on separate Git branches and later combine their work.

3. Git vs GitHub

These two terms are often confused.

Git

Git is the **version control software** running on your computer.

GitHub

GitHub is an **online platform** where Git repositories can be stored and shared.

Think of it like this:

```
Git      → Tool
GitHub   → Online service/platform
```

Other Git hosting platforms include GitLab and Bitbucket.

4. What is a Repository?

A **repository**, or **repo**, is a project that Git is tracking.

It contains:

```
Project Files
+
```

```
Git History
+
Branches
+
Commits
```

For example:

```
my-website/
|
├─ index.html
├─ style.css
├─ script.js
└─ .git/
```

The `.git` directory contains Git's internal information and history.

Important: Do not manually modify the `.git` directory.

5. Creating a Git Repository

Suppose we create a project:

Bash

```
mkdir my-project
cd my-project
```

Now initialize Git:

Bash

```
git init
```

You may see something similar to:

```
Initialized empty Git repository
```

Git has now started tracking the project.

You can check the status:

Bash

```
git status
```

6. The Git Workflow

One of the most important concepts in Git is the workflow.

A simplified workflow looks like this:

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Git Repository
```

Let's understand each part.

7. Working Directory

The **working directory** is where you actually create and modify files.

For example:

```
my-project/
├─ index.html
├─ style.css
└─ app.js
```

You edit these files normally.

Git detects that they have changed.

Check:

Bash

```
git status
```

Example:

```
Changes not staged for commit:
  modified: index.html
```

8. Staging Area

The **staging area** is where you select the changes that you want to include in your next commit.

For example:

Bash

```
git add index.html
```

Now `index.html` is staged.

Check:

Bash

```
git status
```

You may see:

```
Changes to be committed:
  modified:   index.html
```

You can stage multiple files:

Bash

```
git add index.html style.css
```

Or all changed files:

Bash

```
git add .
```

Simple analogy

Think of sending a package.

Working Directory

↓

Things lying on your desk

Staging Area

↓

Things you put inside the package

Commit



Package is sealed and recorded

9. What is a Commit?

A **commit** is a saved snapshot of your project at a particular point in time.

You create a commit using:

Bash

```
git commit -m "Add homepage"
```

The `-m` option allows you to provide a message.

Example:

Bash

```
git commit -m "Add login page"
```

The commit records:

- What changed
- When it changed
- Who made the change
- A unique commit ID
- The commit message

10. Why Are Commit Messages Important?

A commit message should explain **what the commit did**.

Good examples

```
Add login page  
Fix payment calculation  
Add user registration  
Update database configuration  
Fix navbar alignment
```

Poor examples

```
changes
update
test
abc
final
```

A good commit history makes it easier to understand the project.

11. Viewing Commit History

Use:

Bash

```
git log
```

Example:

```
commit a82f91c
Author: John
Date: ...
```

Add login page

```
commit 91bd231
Author: John
Date: ...
```

Add homepage

A shorter version:

Bash

```
git log --oneline
```

Example:

```
a82f91c Add login page
91bd231 Add homepage
```

This is very useful for quickly viewing project history.

12. Complete Basic Git Example

Let's create a simple project.

Step 1: Create project

Bash

```
mkdir calculator  
cd calculator
```

Step 2: Initialize Git

Bash

```
git init
```

Step 3: Create a file

Create:

```
calculator.py
```

Add:

Python

```
def add(a, b):  
    return a + b
```

Step 4: Check status

Bash

```
git status
```

Git may show:

```
Untracked files:  
  calculator.py
```

Step 5: Stage the file

Bash

```
git add calculator.py
```

Step 6: Commit

Bash

```
git commit -m "Add calculator function"
```

Now the first version of the project is saved.

13. Making Another Change

Suppose we modify `calculator.py`:

Python

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b
```

Check:

Bash

```
git status
```

Git detects the change.

Stage it:

Bash

```
git add calculator.py
```

Commit it:

Bash

```
git commit -m "Add subtraction function"
```

Now we have:

```
Commit 1  
↓  
Add calculator function  
↓
```

Commit 2

Add subtraction function

This is the basic idea behind Git history.

14. What is a Branch?

A **branch** is an independent line of development.

Branches allow developers to work on different features without immediately changing the main code.

Imagine your project:

```
main
```

— • — • — •

Now you create a feature branch:

```
main
```

— • — • — •

feature

The feature branch can contain changes that are separate from `main`.

15. Why Do We Use Branches?

Suppose your application is already working correctly.

You want to add a new payment feature.

You don't want unfinished payment code directly in the main version.

So you create:

```
main
↓
feature/payment
```

You work on the payment feature there.

```
main
↓
stable application

feature/payment
↓
payment development
```

When the feature is complete, you can merge it into `main`.

16. Creating a Branch

Modern Git:

Bash

```
git switch -c feature-login
```

This:

1. Creates a branch
2. Switches to that branch

Another commonly used command is:

Bash

```
git checkout -b feature-login
```

Both are commonly encountered, but `git switch` is more focused on branch switching.

17. Viewing Branches

Use:

Bash

```
git branch
```

Example:

```
* feature-login  
main
```

The `*` indicates the branch you are currently using.

18. Switching Branches

To switch to `main`:

```
Bash  
git switch main
```

To switch back:

```
Bash  
git switch feature-login
```

Older projects/tutorials may use:

```
Bash  
git checkout main
```

19. Branch Example

Suppose we have:

```
main
```

We create:

```
Bash  
git switch -c feature-login
```

Now:

```
main
```

```
\  
feature-login
```

We create `login.html`.

Then:

Bash

```
git add login.html  
git commit -m "Add login page"
```

The feature branch now contains the login feature.

20. What is Merging?

Merging means combining changes from one branch into another branch.

For example:



After the login feature is complete, we want it in `main`.

First switch to `main`:

Bash

```
git switch main
```

Then merge:

Bash

```
git merge feature-login
```

Git combines the changes.

Conceptually:

```
feature-login  
↓
```

|
▼
main

21. Complete Branch and Merge Example

Imagine an application has:

main

Create a feature:

Bash

```
git switch -c feature-login
```

Create:

login.html

Commit:

Bash

```
git add login.html  
git commit -m "Add login page"
```

Now go back to main:

Bash

```
git switch main
```

Merge:

Bash

```
git merge feature-login
```

The login feature is now part of `main`.

22. What is a Merge Conflict?

Sometimes Git cannot automatically combine changes.

This is called a **merge conflict**.

For example, suppose `main` contains:

```
Welcome to our website
```

And the feature branch changes the same line to:

```
Welcome to our application
```

At the same time, another developer changes it in `main` to:

```
Welcome to our online store
```

Git doesn't know which version you want.

It may mark the file like this:

```
<<<<<<< HEAD
Welcome to our online store
=====
Welcome to our application
>>>>>>> feature-login
```

You must manually decide which version should remain.

For example:

```
Welcome to our online application store
```

Then:

Bash

```
git add .
git commit -m "Resolve merge conflict"
```

23. Understanding HEAD

`HEAD` is a reference to the commit/branch you are currently working on.

For example:

```
main
↓
Commit A
↓
Commit B
↓
Commit C
↑
HEAD
```

If you switch to another branch:

```
feature-login
↓
Commit D
↑
HEAD
```

So you can think of `HEAD` as:

"Where am I currently working?"

24. Git Status

One of the most useful commands is:

Bash

```
git status
```

It tells you things such as:

- Current branch
- Modified files
- Staged files
- Untracked files
- Whether your branch is ahead/behind another branch

Use it frequently.

25. Git Diff

To see what has changed but has not been staged:

Bash

```
git diff
```

Example:

Diff

```
- Hello World  
+ Hello Git
```

This means:

```
- old line  
+ new line
```

To see staged changes:

Bash

```
git diff --staged
```

26. Untracked Files

Suppose you create:

```
notes.txt
```

Git has not started tracking it yet.

`git status` might show:

```
Untracked files:  
  notes.txt
```

To start tracking it:

Bash

```
git add notes.txt
```

Then commit:

Bash

```
git commit -m "Add project notes"
```

27. Git Add vs Git Commit

This is an important distinction.

```
git add
```

Selects changes for the next commit.

Bash

```
git add file.txt
```

```
git commit
```

Creates a permanent snapshot in Git history.

Bash

```
git commit -m "Update file"
```

Think:

```
Edit file
  ↓
git add
  ↓
Prepare changes
  ↓
git commit
  ↓
Save snapshot
```

28. Local Repository vs Remote Repository

A repository can exist on your computer and also on a remote server.

Local

Your Computer

↓

Local Git Repository

Remote

GitHub / GitLab / Bitbucket

↓

Remote Repository

A common workflow is:

Local changes

↓

`git add`

↓

`git commit`

↓

`git push`

↓

Remote repository

29. Git Clone

If a repository already exists online, you can download it using:

Bash

```
git clone <repository-url>
```

For example:

Bash

```
git clone https://example.com/my-project.git
```

This creates a local copy of the repository.

You can then:

Bash

```
cd my-project
```

and start working.

30. Git Push

`git push` sends your local commits to the remote repository.

Example:

Bash

```
git push origin main
```

Conceptually:

```
Your computer
  |
  | git push
  ▼
Remote repository
```

31. Git Pull

`git pull` gets changes from the remote repository and integrates them into your current branch.

Bash

```
git pull
```

Conceptually:

```
Remote repository
  |
  | git pull
  ▼
Your local repository
```

A common team workflow is:

```
git pull
↓
Make changes
↓
git add
↓
git commit
↓
```

32. Important Git Commands

Command	Purpose
<code>git init</code>	Create a Git repository
<code>git clone</code>	Copy a remote repository
<code>git status</code>	Show current Git status
<code>git add</code>	Stage changes
<code>git commit</code>	Save a snapshot
<code>git log</code>	View commit history
<code>git diff</code>	View changes
<code>git branch</code>	View/manage branches
<code>git switch</code>	Switch branches
<code>git merge</code>	Merge branches
<code>git pull</code>	Get remote changes
<code>git push</code>	Send commits to remote
<code>git remote -v</code>	View remote repositories

33. Complete Real-World Example

Let's imagine you are developing an e-commerce website.

Initially:

```
main
```

You create the basic website:

Bash

```
git add .  
git commit -m "Create basic website"
```

Now you need a login feature.

Create a branch:

Bash

```
git switch -c feature-login
```

Work on the login feature.

Then:

Bash

```
git add .  
git commit -m "Add user login"
```

The main branch is still unchanged.

Now switch back:

Bash

```
git switch main
```

Merge the feature:

Bash

```
git merge feature-login
```

Now:

```
graph LR
    main((main)) --- Basic[Basic website]
    main --- Login[Login feature]
```

feature-login
/
Basic Login
website feature

The login feature is now included in `main`.

34. Recommended Branch Naming

Good branch names make projects easier to understand.

Examples:

```
feature/login  
feature/payment  
feature/user-profile  
  
bugfix/login-error  
bugfix/payment-calculation  
  
hotfix/security-issue
```

A common pattern is:

```
feature/<feature-name>  
bugfix/<bug-name>  
hotfix/<issue-name>
```

35. Good Git Practices

1. Make small commits

Instead of:

```
Add login + payment + dashboard + database
```

Prefer:

```
Add login page  
Add login validation  
Add payment page  
Add dashboard
```

2. Write meaningful commit messages

Good:

```
Fix login validation
```

Not so good:

```
update
```

3. Create branches for features

Instead of doing everything directly on `main`:

```
main
```

Use:

```
main
├── feature/login
├── feature/payment
└── feature/dashboard
```

4. Check status frequently

Bash

```
git status
```

5. Pull before starting work

When working with a team:

Bash

```
git pull
```

This helps you start with the latest remote changes.

36. A Simple Mental Model

Remember Git using this flow:

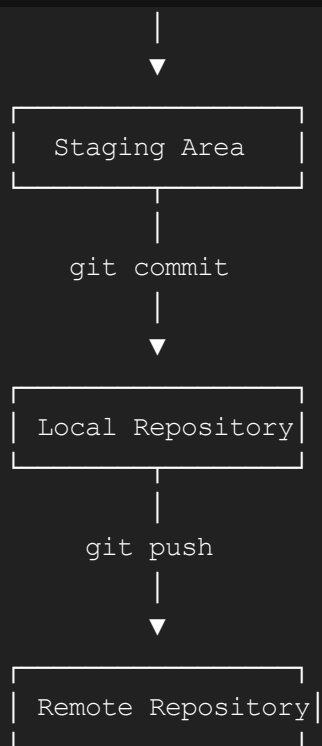
Git Workflow

Edit Files

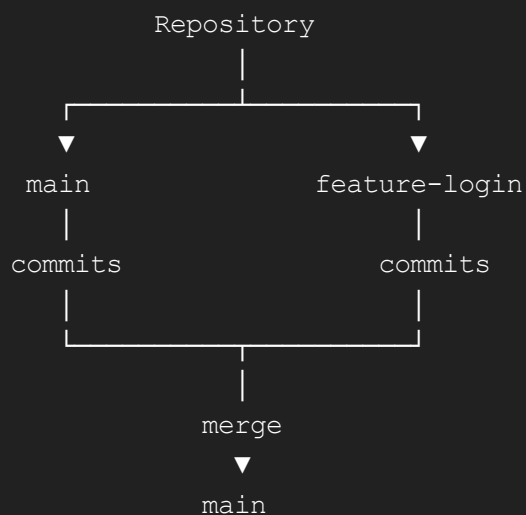


Working Directory

git add



Branches sit on top of this workflow:



37. Beginner Practice Exercise

Try this exercise on your computer.

Exercise: Create a Student Management Project

Create a project:

Bash

```
mkdir student-management
cd student-management
```

```
git init
```

Create:

```
README.md  
students.txt
```

Add and commit:

Bash

```
git add .  
git commit -m "Create student management project"
```

Now create a branch:

Bash

```
git switch -c feature-student-registration
```

Add some information to `students.txt`:

```
John  
Alice  
Robert
```

Commit:

Bash

```
git add students.txt  
git commit -m "Add student registration"
```

Switch to main:

Bash

```
git switch main
```

Merge:

Bash

```
git merge feature-student-registration
```

Check history:

Bash

```
git log --oneline
```

You have now practiced:

Repository

↓

Files

↓

Staging

↓

Commit

↓

Branch

↓

Changes

↓

Merge

38. Quick Interview Questions

What is Git?

Git is a distributed version control system used to track changes in files and manage different versions of a project.

What is a repository?

A repository is a project tracked by Git, including its files and version history.

What is a commit?

A commit is a snapshot of changes saved in Git history.

What is a branch?

A branch is an independent line of development used to work on features or fixes without directly changing another branch.

What is merging?

Merging combines changes from one branch into another branch.

What is the staging area?

The staging area contains changes selected for the next commit.

What does `git status` do?

It shows the current state of the working directory and staging area.

What does `git push` do?

It sends local commits to a remote repository.

What does `git pull` do?

It retrieves changes from a remote repository and integrates them into the current local branch.

What is a merge conflict?

A merge conflict occurs when Git cannot automatically determine how to combine conflicting changes.

39. The Most Important Commands to Remember

If you are just starting with Git, focus on these first:

Bash

```
git init
git status
git add .
git commit -m "message"

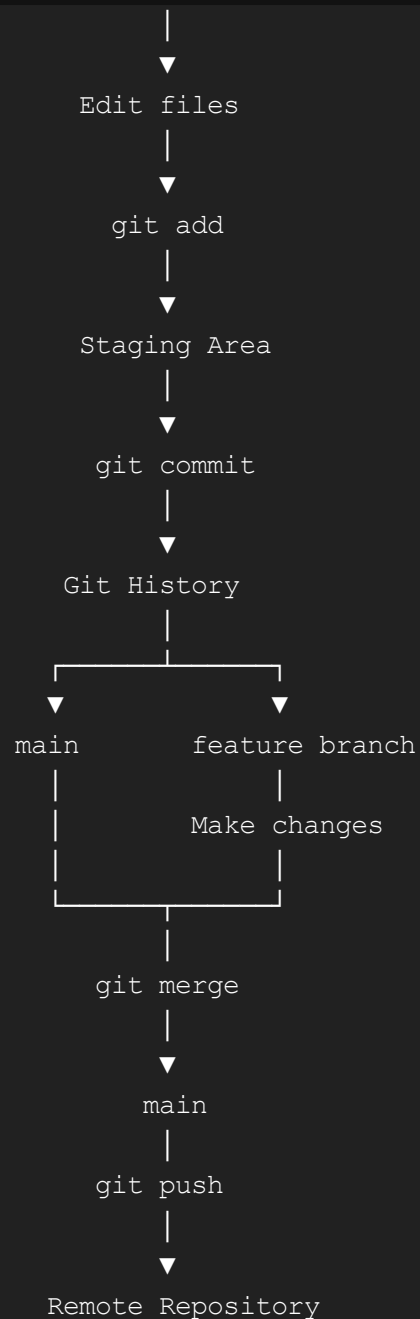
git branch
git switch -c feature-name
git switch main
git merge feature-name

git log --oneline

git clone <url>
git pull
git push
```

The basic pattern

```
      GIT
    git init / clone
      |
      ▼
  Repository
```



In one sentence: Git lets you **track changes**, commits let you **save snapshots**, branches let you **work independently**, and merges let you **combine that work**.

Log in for more personalized help with writing, rewriting, and translation.

Log in

Sign up for free